19 October 2025 07:51 AM

Introduction to Competitive Programming:

Competitive Programming (CP) is a mind-sport where participants solve algorithmic and mathematical problems using programming languages like C++, Python, or Java. It tests logic, problem-solving skills, and coding efficiency under time constraints—helping you master data structures, algorithms, and analytical thinking essential for tech interviews and real-world software development.

Just like chess you can earn ratings based on your performance in contests,
with platforms like Codeforces and CodeChef where you compete and improve your skills.

Sristi, our College Techfest, is organizing not one but two Competitive Programming contest with special prizes exclusively for 1st-year students — it's highly recommended to participate and showcase your problem-solving skills!

Why you should prefer C++

(For Competitive Programming)

- Efficiency and Speed
- Most popular language for CP
- In-built Data Structures and Algorithms (STL)

Goals:

- Variables and Datatypes
- Input / Output
- Different Types of Operations
- Conditional Statements (If-Else Statements)
- Loops
- Arrays
- String
- Functions
- Tricks for Competitive Programming
- Basic C++ Template for CP
- Problem Solving in Codeforces

Common Skeleton of a C++ code:

```cpp
#include <iostream>
using namespace std;

int main(){
    // code here

    return 0;
}
```

Simplest C++ Program:

```cpp
#include <iostream>
using namespace std;

int main(){
    cout << "Hello World!";

    return 0;
}
```

Output

Hello World!

19 October 2025 07:54 AM

Variables:

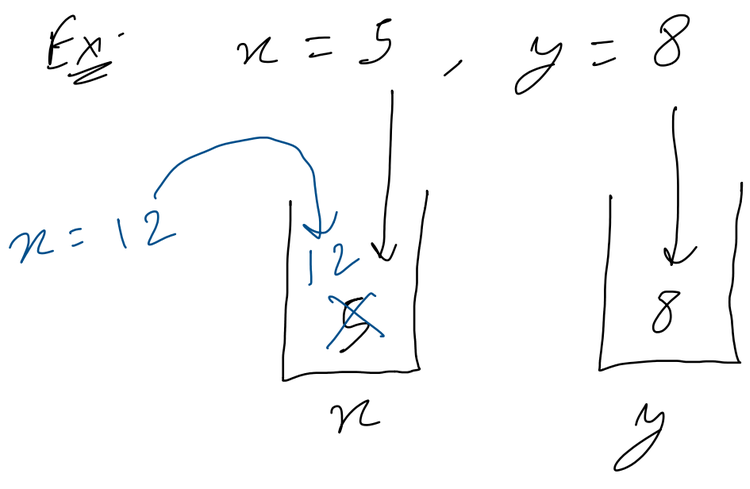
just like x,y,z in maths equations

It is a named memory location that holds a value,
that can be changed during a program's execution.
Think of it as a container with a label for storing data like numbers or texts.

Variable names cannot:

- Have spaces (use underscore instead)
- Start with a digit
- Be reserved by the compiler (Keywords not allowed)
- Already taken by another variable (in the same scope)

NOTE: Keywords/Variables are case sensitive



Datatypes: used to set the "type" of a variable. For example, int is used to declare integer variables.

(i) *Integer Datatype:* used to store integer contants inside a variable, like 4 | 64 | -90

Type	Bytes	Range of Value	Example
int	4	-10^9 to 10^9	int x = 10;
long	4	-10^12 to 10^12	long x = -15;
long long	8	-10^18 to 10^18	long long x = 150000;

NOTE: In CP, we almost always use **long long** to handle large integers and avoid overflow.
So we can predefine int as long long, so that even if we write int it works like long long.

```
#define int long long
```

(ii) *Decimal Datatype:* used to store decimal constants inside a variable, like 3.14 | 12.0 | 0.3333

Type	Digits	Example
float	7	float x = 5.6;
double	15	double x = 8.97;
long double	18	long double x = 2.5;

NOTE: Make sure you don't print big numbers with floating-point datatypes

(iii) *Character Datatype:* used to store a single char inside a variable, like 'f' | '5' | '~' | '\n'
Ex: char ch = 'h';

(iv) *String Datatype:* used to store string literal inside a variable, like "Hello" | "My Name is"
Ex: string s = "Hey Anish";

NOTE: `getline(cin, s); // used to read string with spaces`

19 October 2025 08:32 AM

Output in C++

To output a value, we use the **cout** operator as follows: cout << value;

To print multiple values in the same line:
cout << value1 << value2 << value3;

To start printing in a new line: **endl** or **'\n'**

Input in C++

To input a value, we use the **cin** operator as follows: cin >> value;

To take multiple values as input in the same line:
cin >> value1 >> value2 >> value3;

NOTE: Each input value must be separated by a space or a new line.

Code:

```
int main(){
    int x,y;
    cin >> x >> y;
    cout << "Value of x: " << x << "and y: " << y;

    return 0;
}
```

Input

10 12 or 10
12

Output

Value of x : 10 and y : 12

Operators

19 October 2025 09:03 AM

Arithmetic operators in C++ :

1. + Addition
2. - Subtraction
3. * Multiplication
4. / Division (Quotient)
5. % Modulo (Remainder)

NOTE: C++ follows the **BODMAS** rule

Example:

```
int main(){
    int x=5;
    int y=8;

    int sum=x+y;
    int diff=x-y; //this will be negative as x<y
    int product=x*y;
    int division=x/y; //this should be y/x for correct division as x<y
    int remainder=x%y; //this should be y%x to get correct remainder

    cout<<sum<<' '<<diff<<' '<<product<<'\n';
    cout<<division<<' '<<remainder;
    cout<<1+1*2<<' '<<(1+1)*2; // you have to use brackets to change precedence

    return 0;
}
```

Output:

```
13 -3 40
0 5
3 4
```

Relational Operation:

==	check for equal
>	left is greater than right
<	right is greater than left
>=	left is either equal or more than right
<=	right is either equal or more than left
!=	left is not equal to right

Logical operators

Logical operators perform operations on boolean values or expressions that result in Boolean values.

1. “(expr1) && (expr2)” checks whether BOTH are true.
2. “(expr1) || (expr2)” checks whether EITHER one is true.
3. “!(expr)” returns the OPPOSITE of the result of “expr”

The operators are called AND, OR, NOT operators respectively

Conditional Statement

19 October 2025 11:54 AM

If - Else Statement :

Syntax:

```
if (condition) {
    // code
} else if (another_condition) {
    // code
} else {
    // code
}
```

Ex:

```
int main(){
    int n;
    cin>>n;
    if(n>12){
        cout<<"Greater than 12"<<endl;
    }else if(n>9){
        cout<<"Greater than 9"<<endl;
    }else{
        cout<<"No"<<endl;
    }
    return 0;
}
```

Loops

19 October 2025 04:51 PM

“for” loop

Syntax:

```
for (s1; s2; s3) {
    // Code here
}
```

s1: Executed once before start of loop.

s2: Condition of the loop. Loop exits if false.

s3: Executed after each iteration.

```
int main(){
    int n;
    cin >> n;
    for(int i = 0; i < n; ++i){
        cout << "Sristi!" << endl;
    }
    return 0;
}
```

Input

5

Output

Sristi!
Sristi!
Sristi!
Sristi!
Sristi!

“while” loop

Syntax:

```
while (    ) {
    // Code here
}
```

Check if the condition is true and then execute the block of code. Repeat.

```
int main(){
    int n;
    cin >> n;
    int i=0;
    while(i < n){
        cout << "Sristi!" << endl;
        i++;
    }
    return 0;
}
```

Input

5

Output

Sristi!
Sristi!
Sristi!
Sristi!
Sristi!

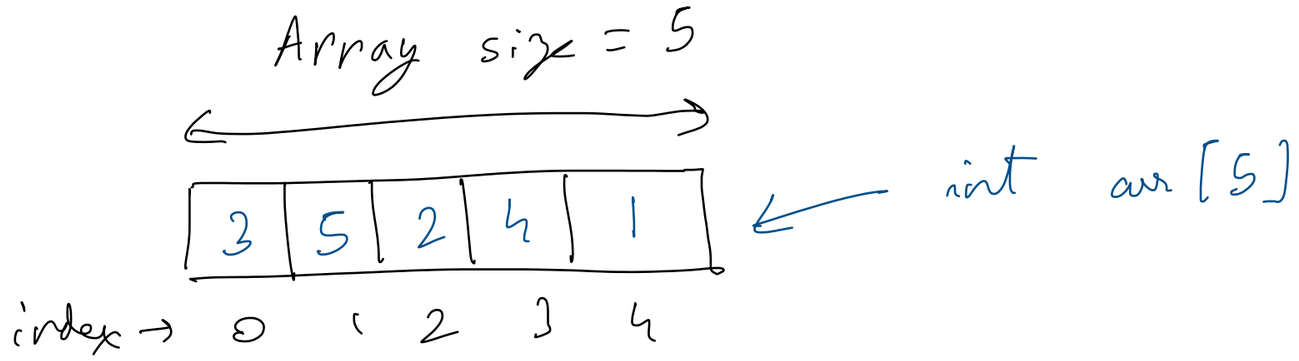
19 October 2025 04:55 PM

Arrays

An array is a collection of multiple items of the same datatype.

- Arrays are ordered.
- The size of an array cannot be changed.

Syntax: `datatype name[size]`



Taking Input of a array of size n:

```cpp
int main() {
    int n;
    cin >> n;

    int arr[n];
    for(int i=0; i<n; i++) {
        cin >> arr[i];
    }

    return 0;
}
```

19 October 2025 04:55 PM

Functions

Functions are reusable blocks of code that can be run whenever called.

They can take in parameters (input) and return a value (output).

Syntax:

```
return_type name(d1 param1, d2 param2, …) {
    // result must be same as return_type
    return result;
}
```

Void Function:

```
#include <iostream>
using namespace std;

void modify(int a) {
    a = a + 10;
    cout << a << endl; // Output: 15
}

int main() {
    int x = 5;
    modify(x);
    cout << x << endl; // Output: 5

    return 0;
}
```

Return Function:

```
#include <iostream>
using namespace std;

int modify(int a) {
    a = a + 10;
    return a; // return the modified value 15
}

int main() {
    int x = 5;
    modify(x);
    cout << x << endl; // Output: 15

    return 0;
}
```

Tricks for CP

19 October 2025 04:56 PM

Header file for Competitive Programming

```
#include <bits/stdc++.h>
```

Pros:

- Includes every standard library and STL headers. Therefore, it saves time spent coding during contests.

Cons:

- Increases compile time (Doesn't matter for CP)
- Does not work with compilers other than GNU C++

Fast I/O

```
ios::sync_with_stdio(false);
```

Removes sync between cout and printf.

```
cin.tie(NULL);
```

Removes sync between cout and cin.

```
endl vs '\n'
```

"endl" forces the input buffer to flush.
When using fastio, use '\n' rather than endl

Basic C++ Template for CP

19 October 2025 04:56 PM

```cpp
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define int long long

int32_t main(){
    ios::sync_with_stdio(false); cin.tie(nullptr);

    int tc; cin >> tc; // if there are multiple testcases
    while(tc--){
        // code here
    }

    return 0;
}
```

07 September 2025 09:09 PM

A. Watermelon

time limit per test: 1 second
memory limit per test: 64 megabytes

One hot summer day Pete and his friend Billy decided to buy a watermelon. They chose the biggest and the ripest one, in their opinion. After that the watermelon was weighed, and the scales showed w kilos. They rushed home, dying of thirst, and decided to divide the berry, however they faced a hard problem.

Pete and Billy are great fans of even numbers, that's why they want to divide the watermelon in such a way that each of the two parts weighs even number of kilos, at the same time it is not obligatory that the parts are equal. The boys are extremely tired and want to start their meal as soon as possible, that's why you should help them and find out, if they can divide the watermelon in the way they want. For sure, each of them should get a part of positive weight.

Input

The first (and the only) input line contains integer number w ($1 \leq w \leq 100$) — the weight of the watermelon bought by the boys.

Output

Print `YES`, if the boys can divide the watermelon into two parts, each of them weighing even number of kilos; and `NO` in the opposite case.

Examples

input	Copy
8	
output	Copy
YES	

Note

For example, the boys can divide the watermelon into two parts of 2 and 6 kilos respectively (another variant — two parts of 4 and 4 kilos).

Solution:

check if the watermelon's weight (w) is an even number and greater than 2.
If w is even and greater than 2, it can be split into two even parts (e.g., w=8 can be split into 4 and 4).
If w is odd or if w is equal to 2, it is impossible to split it into two even parts, so the answer is "NO"

Code:

```cpp
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define int long long

int32_t main(){
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n;
    cin>>n;

    if(n>2){
        if(n%2==0){
            cout<<"YES";
        }else{
            cout<<"NO";
        }
    }
    else{
        cout<<"NO";
    }

    return 0;
}
```